

# Build your own weather MCP server

**Grade: B**

Status: Pass (automatic by LLM)

## Feedback

Hard blockers (automatic failure conditions)

- None detected. No hardcoded secrets, a new weather MCP server is present with 3 tools, and it does not reuse the Alpaca server.

Grading table

### 1. MCP Server Correctness (30)

- 1.a Three distinct tools exposed: 10/10 -- `get_current_weather`, `get_forecast`, `predict_umbrella_needed` are decorated with `@mcp.tool` in `weather_mcp_server.py`.
- 1.b Tool docstrings with Args/Returns style: 2/5 -- Docstrings are present but are one-liners without Args/Returns sections (e.g., "Get current weather for a city or place.").
- 1.c HTTP/parsing logic in separate adapter: 5/5 -- `weather_broker.py` contains all HTTP calls and response shaping; tools call a service layer.
- 1.d Streamable HTTP server/transport suitable for Apps: 5/5 -- `mcp.run` uses `transport="streamable-http"` and `app.yaml` is provided.
- 1.e Reasonable error handling: 5/5 -- Tools catch `ValueError/OSError` and return `{"error": "..."};` inputs validated in `WeatherService`.

Score: 27/30

### 2. Prediction/Recommendation Logic (15)

- 2.a Derived logic beyond pass-through: 10/10 -- `should_bring_umbrella` uses nonzero precipitation or probability ? 50%.
- 2.b Logic explained in docstring/README: 5/5 -- `README.md` documents the threshold explicitly.

Score: 15/15

### 3. Secrets & Security (15)

- 3.a No hardcoded API keys/secrets: 10/10 -- Open-Meteo requires none; no secrets found in code.
- 3.b If API requires key, fetched from secret store: 5/5 -- Not applicable; project correctly avoids adding secrets when API doesn't require them.

Score: 15/15

### 4. Agent Configuration (20)

- 4.a Agent registered against your MCP server as an external tool: 0/5 -- No agent tool configuration (JSON/YAML) or screenshot proving registration; evidence only shows the App running.
- 4.b System prompt instructs usage order/scope: 4/5 -- Strong guardrails and scope; not an explicit "call forecast before predicting," though `predict_umbrella_needed` encapsulates that.
- 4.c At least one explicit guardrail: 5/5 -- Multiple, including "don't invent

data," "ask for clarification," "identify Open-Meteo," "never claim a tool was used if it wasn't."

- 4.d Agent behavior matches system prompt: 2/5 -- Minimal transcript summaries (04\_agent\_bricks\_conversations.txt) don't show attribution to Open-Meteo or clarification steps; insufficient to verify alignment beyond tool names.

Score: 11/20

#### 5. Documentation & Deployment Readiness (10)

- 5.a README explains architecture, tool list, API + auth: 5/5 -- README.md explains pipeline, tools, Open-Meteo choice, and no-credentials stance.

- 5.b requirements/app manifest plausible for Databricks App: 5/5 -- requirements.txt and app.yaml present; 03\_remote\_services.txt shows the App running.

Score: 10/10

#### 6. Demonstration (10)

- 6.a Three distinct NL questions with tool calls and final answers: 0/5 -- Provided entries summarize results but don't include the actual natural-language questions or full transcripts/screenshots of the conversations.

- 6.b Answers consistent with tool outputs: 4/5 -- Values shown are plausible and correspond to tool names, but without full traces it's hard to fully verify.

Score: 4/10

Total score: 82/100

#### Most important improvements

- Provide concrete Agent Bricks tool configuration evidence: the agent definition JSON/YAML showing your MCP endpoint registered, plus a screenshot of the Agent Bricks UI or a logged tool-call trace proving the agent used your external MCP tools.

- Expand demonstration artifacts into real transcripts: include the user's exact natural-language questions, the tool call traces (inputs/outputs), and the final answers; add at least one ambiguous-location example and one error-path example.

- Upgrade tool docstrings to include Args>Returns and types for each parameter/field, matching the alpaca\_mcp\_server.py style.

- Add a small negative-path unit test and/or curl repro showing clean error responses (e.g., geocoding failure) propagating through the MCP tools.

- Consider logging minimal structured traces (tool name, input, outcome, duration) to help validate agent behavior and attribution to Open-Meteo.

#### Notes on submission packaging

- README\_BUILD\_YOUR\_OWN\_MCP.md mainly covers a stock-market assistant; this is confusing for a weather MCP submission. Please link prominently from that doc to the weather MCP section and include the referenced docs/EVIDENCIAS\_WEATHER.md (it's missing here). Consolidating or clearly separating the weather deliverables would make grading easier.

If anything here is missing due to my request setup, please provide:

- The Agent Bricks tool configuration (JSON/YAML) or a screenshot from the Agent UI showing your weather MCP server registered and enabled.

- Full conversation transcripts (or screenshots) with three natural-language questions, including the tool-call logs and final answers (showing Open-Meteo

attribution).

- The docs/EVIDENCIAS\_WEATHER.md file referenced in your README.
- Any deployment URL(s) for your Databricks App(s), or CLI output showing the MCP server's public endpoint details.